

Part 2 — Methodology and Findings Report

Methodology

The Part 2 pipeline takes the raw Metro Interstate Traffic Volume CSV (48,204 rows) and turns it into an ML-ready dataset through four stages, each in its own script and each fully logged (see `pipeline.log`):

1. ``pipeline.py`` loads the CSV with explicit file-handling error handling (no bare `except :`), validates the schema against the 9 expected columns *before* any other processing, then cleans the data: standardises inconsistent `weather_description` casing (e.g. "Sky is Clear" vs "sky is clear", and an all-caps "SQUALLS" outlier), parses and validates `date_time`, removes 17 exact duplicate rows and collapses 7,612 rows that shared a timestamp with another row (a known artefact of the source weather API logging more than one condition per hour) down to one row per hour, and imputes two types of physically-impossible sensor readings, a 0 Kelvin temperature (10 rows) and a >9,000mm hourly rainfall reading (1 row), using **that calendar month's median** of the valid readings, computed with an explicit loop over the 12 months rather than one global average, since weather varies enormously by season.
2. ``feature_engineering.py`` builds the ML-ready feature set: `hour`, `day_of_week`, `is_weekend`, and a sine/cosine cyclical encoding of both hour and day-of-week (so hour 23 and hour 0 are correctly treated as adjacent); one-hot encoded weather categories plus derived `is_severe_weather` / `is_low_visibility` / `has_precipitation` indicators; min-max scaled versions of `temp` and `clouds_all`; and a data-driven `congestion_category` (Low/Medium/High/Severe) based on the quartiles of `traffic_volume`, reused again in Part 3 for the proxy accident-risk label.
3. ``visualizations.py`` produces four Matplotlib charts (traffic by hour, weekday vs. weekend, traffic volume distribution, and average traffic by weather condition), each saved to `figures/` with a short written interpretation in `figures/figure_interpretations.md`.
4. ``cli_app/app.py`` exposes the processed dataset through four commands (`query-datetime`, `high-traffic`, `compare-weekday-weekend`, `recommend-travel`) so a non-technical user can query it directly from the command line. Each command validates its own input (an invalid `--datetime`, a negative `--top`, or an unrecognised `--day-type` all produce a clean error message and a logged ERROR line rather than a raw traceback) — see Sample CLI Output below.

`main.py` runs all three pipeline stages in one process, producing one continuous, correctly-ordered `pipeline.log` (the version committed to this repo).

Findings

- **Data quality:** the raw file is generally well-formed but contains three distinct real-world data issues, each handled explicitly rather than silently: casing inconsistencies in one text column, duplicate/overlapping timestamp rows, and a small number of physically impossible sensor readings. After cleaning, 40,575 of the original 48,204 rows remain as one-row-per-hour records (most of the "lost" rows are the duplicate-timestamp collapse, not discarded information — the duplicate weather tag for an already-recorded hour was dropped, not the hour itself).
- **Time-of-day dominates.** The hourly average traffic chart shows a sharp bimodal commute pattern; the weekday-vs-weekend comparison shows weekdays have two distinct peaks that weekends do not, confirming (consistently with the Part 1 SQL/probability analysis) that time and day-type are far stronger drivers of traffic volume than weather.

- **The traffic volume distribution is bimodal**, not a simple bell curve — a low-traffic overnight cluster and a separate moderate/high daytime cluster — which is why single summary statistics (Part 1, Task 2) need to be read alongside the hourly pattern rather than in isolation.
- **Weather's effect is real but modest**: the highest- and lowest-average- traffic weather categories differ by a moderate margin, but the spread across weather categories is small next to the hour-of-day effect, reinforcing the Part 1 finding that weather is a weak lever for predicting congestion on this corridor.

Sample CLI Output

Actual runs of `cli_app/app.py` against the processed dataset (log timestamps trimmed for readability; full trail in `pipeline.log`):

```
> python app.py high-traffic --top 5
Top 5 highest-traffic hours of day (by average volume):
16:00 -> 5709 vehicles/hour
17:00 -> 5351 vehicles/hour
15:00 -> 5272 vehicles/hour
14:00 -> 4957 vehicles/hour
07:00 -> 4770 vehicles/hour

> python app.py query-datetime --datetime "2017-07-04 08:00:00"
Date/time      : 2017-07-04 08:00:00
Traffic volume : 1333 vehicles/hour (Medium)
Weather        : Clear (sky is clear)
Temperature    : 20.3°C
Day type       : Weekday

> python app.py compare-weekday-weekend
Average weekday traffic : 3557 vehicles/hour
Average weekend traffic  : 2624 vehicles/hour
Difference              : +934 vehicles/hour (+35.6% vs. weekend)

> python app.py recommend-travel --day-type weekend
Recommended low-traffic travel windows for a weekend journey:
06:00-07:00 -> ~1103 vehicles/hour (historically light)
07:00-08:00 -> ~1600 vehicles/hour (historically light)
08:00-09:00 -> ~2354 vehicles/hour (historically light)
```

For a weekend journey, consider travelling between 06:00 and 07:00, when historical traffic volumes are typically lower.

Input validation, demonstrated deliberately with bad input rather than happy-path values only:

```
> python app.py high-traffic --top -3
ERROR    | Invalid --top value received: -3 (must be a positive integer)
Error: --top must be a positive integer.

> python app.py query-datetime --datetime "not-a-date"
ERROR    | Invalid --datetime value received: 'not-a-date' (expected e.g. '2017-07-04 08:00:00')
Error: 'not-a-date' is not a valid date/time. Expected format: YYYY-MM-DD HH:MM:SS
```

Both cases exit cleanly with a user-facing error message and a logged `ERROR` entry, rather than an unhandled traceback.

Note: `query-datetime` reports July 4, 2017 as "Weekday" — `is_weekend` only checks whether the calendar day is Saturday/Sunday, not whether it's a federal holiday, so a weekday holiday like Independence Day is correctly flagged by its own `holiday` column elsewhere in the dataset but still counts as a "Weekday" for this specific day-type comparison. This is expected behaviour, not a bug.

Reproducibility

The entire pipeline runs end-to-end from the raw CSV with `python main.py`, with `requirements.txt` (repo root) pinning every dependency, and Git history showing each task committed incrementally rather than as one final commit (see `git log`).